

ENTREGA PRIMER CORTE



Universidad
del Cauca

Vigilada Mineducación

Ingeniería de Software 2

Entregable del proyecto de curso Banco de preguntas para las pruebas Saber Pro

Presentado por:

Juan Pablo Hernandez

Joseph David Trujillo Gomez

Juan David Meza Paz

Profesor:

Libardo Pantoja

Universidad del Cauca

Facultad de ingeniería electrónica y de telecomunicaciones

Ingeniería de sistemas

Popayán, septiembre de 2026

CONTENIDO

Introducción	2
Historias de usuario implementadas en la primera iteración	1
1. HU01 — Gestión de usuarios del sistema	1
1.1. Criterios de aceptación implementados.....	1
1.2. Evidencia en el código.....	1
1.3. Alcance de usuarios y roles.....	2
2. HU02 — Gestión del banco de preguntas.....	2
2.1. Criterios de aceptación implementados.....	2
4. HU03 — Validación estructural.....	4
4.1. Criterios de aceptación implementados.....	4
4.2. Evidencia en el código.....	4
4.3. Requisitos de validación no implementados específicamente.....	5
5. Prototipos de interfaz y evaluación de usabilidad	5
6.1. Contexto	6
6.2. Estímulo	6
6.3. Respuesta del sistema	6
6.4. Medición de calidad.....	7
6.5. Resultado esperado	7
7. Arquitectura y diseño de software usando C4	8
8. Patrones de diseño implementados.....	11
8.1. MVC	11
8.2. Builder	12
8.3. Strategy	12
8.4. Observer.....	12
8.5. Repository	12
8.6. Dependency Inversion.....	13
9. Requerimientos funcionales y evidencia de cumplimiento	13
10. Requerimientos no funcionales y evidencia de cumplimiento	16
11. Pruebas automatizadas relacionadas con la arquitectura	17
11.1. Validación.....	17
11.2. Dominio	18
11.3. Servicio de negocio	18
11.4. Persistencia.....	19

Introducción

Esta primera entrega del proyecto banco de preguntas del curso Ingeniería de software 2, esta compuesta por diferentes criterios para su entrega.

Entre los criterios están el cumplimiento de los requisitos funcionales, Cumplimiento de los requisitos no funcionales de modificabilidad, documentación de arquitectura, Diagrama C4, y las pruebas unitarias del programa a entregar.

En este archivo se podrán ver diferentes cosas del programa y sus requerimientos, como:

- Historias de usuario implementadas: Cuya teoría fue dada en la materia ingeniería de software 1
- Prototipos de interfaz de usuario y test de evaluación de usabilidad
- Planificación a través de software diseñado para seguir la planeación del marco de trabajo SCRUM o siguiendo la metodología ágil.
- Escenario de calidad modificabilidad: contexto, estímulo, respuesta, medición de calidad, resultado esperado.
- Arquitectura y diseño de software usando el modelo C4

Historias de usuario implementadas en la primera iteración

1. HU01 — Gestión de usuarios del sistema

1.1. Criterios de aceptación implementados

1. El sistema permite autenticar usuarios mediante correo y contraseña.
2. El sistema identifica el rol del usuario autenticado.
3. Las opciones visibles y las operaciones disponibles dependen del rol.
4. Las contraseñas no se almacenan directamente; se almacenan como valores derivados mediante PBKDF2.
5. El usuario puede cerrar sesión e iniciar sesión con otra cuenta.

1.2. Evidencia en el código

- [src/main/java/com/bancopreguntas/domain/Usuario.java](#)
 - Representa la entidad de usuario.
 - Contiene id, nombre, email, rol y passwordHash.
 - El atributo rol identifica al usuario como **AUTOR**, **ADMINISTRADOR** o **REVISOR**.
- [src/main/java/com/bancopreguntas/domain/Rol.java](#)
 - Define los roles que existen en esta iteración.
 - Los roles **DOCENTE** y **ESTUDIANTE** no están definidos en la implementación actual.
- [src/main/java/com/bancopreguntas/ui/DialogoLogin.java](#)
 - Presenta los campos de correo y contraseña.
 - **ingresar()** llama al controlador de autenticación.
 - Cuando las credenciales son incorrectas muestra un mensaje y limpia la contraseña.
- [src/main/java/com/bancopreguntas/service/AutenticacionService.java](#)
 - Define el contrato de autenticación.
- [src/main/java/com/bancopreguntas/service/AutenticacionServiceImpl.java](#)
 - Valida que correo y contraseña tengan valores.
 - Busca el usuario mediante [UsuarioRepository](#).
 - Verifica la contraseña usando **PasswordHasher.verificar(...)**.
 - Devuelve **Optional.empty()** cuando las credenciales no son válidas.
- [src/main/java/com/bancopreguntas/service/PasswordHasher.java](#)

- Utiliza PBKDF2WithHmacSHA256.
- Genera una sal aleatoria de 16 bytes.
- Usa 120.000 iteraciones y una clave derivada de 256 bits.
- Compara el hash calculado utilizando ***MessageDigest.isEqual(...)***.
- [src/main/java/com/bancopreguntas/repository/UsuarioRepositorySQLite.java](#)
 - Persiste usuarios en la tabla usuarios.
 - Consulta usuarios por id, correo o rol.
 - Usa *PreparedStatement*.
 - La consulta por correo utiliza *COLLATE NOCASE*.
- [src/main/java/com/bancopreguntas/ui/VentanaPrincipal.java](#)
 - ***reconstruirTabs(...)*** crea las pestañas según el rol:
 - Autor: Crear pregunta y Mis preguntas.
 - Administrador: Asignar revisores.
 - Revisor: Mis revisiones.
 - ***cambiarUsuario()*** permite cerrar la sesión visual actual y autenticar otra cuenta.
- [src/main/java/com/bancopreguntas/service/PreguntaServiceImpl.java](#)
 - No confía únicamente en la interfaz para controlar permisos.
 - ***exigirRol(...)*** verifica el rol recibido y valida que el usuario exista en el repositorio con el mismo rol.

1.3. Alcance de usuarios y roles

La primera iteración implementa los usuarios de demostración de los roles **AUTOR**, **ADMINISTRADOR** y **REVISOR**. Las cuentas se inicializan en ***App.cargarUsuariosIniciales(...)***.

Los roles **DOCENTE** y **ESTUDIANTE**, mencionados en el requisito general, no están incluidos en el enum Rol ni tienen flujos de interfaz o servicios asociados en esta iteración.

2. HU02 — Gestión del banco de preguntas

2.1. Criterios de aceptación implementados

1. Un autor puede crear una pregunta.
2. La pregunta contiene los campos definidos para el modelo Saber Pro.
3. Una pregunta nueva se guarda en estado Borrador.

4. El autor solo puede modificar preguntas propias.
5. Se pueden consultar preguntas propias mediante filtros.
6. La consulta utiliza paginación.
7. El usuario puede visualizar el detalle completo de una pregunta.
8. Una pregunta rechazada puede ser editada y retorna a Borrador.

3. Evidencia en el código

- [src/main/java/com/bancopreguntas/ui/PanelCrearPregunta.java](#)
 - Construye el formulario de creación y edición.
 - Contiene controles para contexto, pregunta directa, cuatro distractores, respuesta correcta, justificación, bibliografía, competencia, tema, subtema y nivel de dificultad.
 - **guardar()** crea el objeto mediante **Pregunta.builder()**.
 - El autor se toma de **SesionActual.getUsuarioActual()**.
- [src/main/java/com/bancopreguntas/domain/Pregunta.java](#)
 - Contiene todos los campos de la pregunta.
 - El estado inicial del Builder es BORRADOR.
 - **actualizarContenido(...)** permite modificar el contenido bajo el control del servicio.
 - **cambiarEstado(...)** controla el cambio de estado y actualiza la fecha de modificación.
 - **asignarRevisores(...)** actualiza la lista de revisores asignados.
- [src/main/java/com/bancopreguntas/ui/PanelListarPreguntas.java](#)
 - Presenta las preguntas del autor en una tabla.
 - Permite filtrar por texto, tema, subtema, nivel de dificultad y estado.
 - Permite navegar entre páginas.
 - **verDetalle()** muestra todos los campos de la pregunta.
 - **editarPregunta()** abre el mismo formulario en modo edición.
 - Las acciones de editar y eliminar se habilitan solo para estados BORRADOR y RECHAZADA.
- [src/main/java/com/bancopreguntas/service/FiltroPregunta.java](#)
 - Modela los criterios opcionales de consulta.
- [src/main/java/com/bancopreguntas/service/ResultadoPaginado.java](#)
 - Transporta los elementos de la página, total de elementos, tamaño de página y total de páginas.

- [src/main/java/com/bancopreguntas/service/PreguntaServiceImpl.java](#)
 - **crearPregunta(...)** valida el rol, el propietario, el estado inicial y el contenido.
 - **listarPreguntasPorAutor(...)** consulta utilizando el id del actor.
 - Aplica filtros de texto, tema, subtema, competencia, dificultad y estado.
 - Calcula la sublista correspondiente a la página solicitada.
 - **actualizarPregunta(...)** valida que el actor sea el autor propietario.
 - Solo permite editar preguntas en **BORRADOR** o **RECHAZADA**.
 - Si la pregunta estaba **RECHAZADA**, limpia los revisores y la cambia a **BORRADOR**.
- [src/main/java/com/bancopreguntas/repository/PreguntaRepositorySQLite.java](#)
 - Guarda la pregunta principal.
 - Guarda los distractores en pregunta_distractores.
 - Guarda las relaciones con revisores en pregunta_revisores.
 - Reconstruye la pregunta completa al consultarla.

4. HU03 — Validación estructural

4.1. Criterios de aceptación implementados

1. El contexto es obligatorio.
2. La pregunta directa es obligatoria.
3. Deben existir exactamente cuatro distractores.
4. Los distractores no pueden estar vacíos.
5. Los distractores deben ser diferentes entre sí.
6. Debe existir una respuesta correcta.
7. La respuesta correcta no puede coincidir con un distractor.
8. La justificación, bibliografía, competencia, tema, subtema, dificultad y autor son obligatorios.
9. Los errores se acumulan y se presentan al usuario antes de guardar.

4.2. Evidencia en el código

- [src/main/java/com/bancopreguntas/validacion/ValidadorPregunta.java](#)
 - Define la abstracción de validación.
- [src/main/java/com/bancopreguntas/validacion/ValidadorEstructuralPregunta.java](#)

- Implementa la estrategia de validación estructural.
- **requerido(...)** verifica los campos de texto obligatorios.
- **validarDistractoresYRespuesta(...)** exige cuatro distractores.
- Utiliza un HashSet para detectar distractores repetidos.
- Normaliza los valores con **trim().toLowerCase()**.
- Comprueba que la respuesta correcta no esté repetida entre los distractores.
- Devuelve todos los errores encontrados en *ResultadoValidacion*.
- *src/main/java/com/bancopreguntas/service/PreguntaServiceImpl.java*
 - **validar(...)** ejecuta la estrategia antes de guardar o actualizar.
 - Si el resultado no es válido, lanza ValidacionException.
 - El repositorio no recibe la pregunta inválida.
- *src/main/java/com/bancopreguntas/ui/PanelCrearPregunta.java*
 - Captura y muestra los mensajes producidos por ValidacionException.

4.3. Requisitos de validación no implementados específicamente

- No existe una regla explícita que rechace las expresiones Todas las anteriores o Ninguna de las anteriores.
- No existe una validación específica de longitud mínima o máxima para los distractores.
- No existe un analizador gramatical que compruebe estructura sintáctica.
- El modelo representa una única cadena respuestaCorrecta, por lo que la unicidad se maneja como una sola respuesta textual, no mediante un conjunto de respuestas.

5. Prototipos de interfaz y evaluación de usabilidad

La implementación de las interfaces se encuentra en las clases Swing siguientes:

- *DialogoLogin.java*: autenticación.
- *VentanaPrincipal.java*: navegación por rol.
- *PanelCrearPregunta.java*: creación y edición.
- *PanelListarPreguntas.java*: consulta, filtros, paginación y acciones.
- *PanelAsignarRevisores.java*: asignación administrativa.
- *PanelMisRevisiones.java*: revisión por pares.
- *PanelGraficoEstados.java*: seguimiento del estado de las preguntas.

La evaluación de usabilidad no está representada por una clase automatizada dentro del código fuente. Los criterios observables en la implementación son:

- Mensajes de validación mostrados en el formulario.
- Acciones habilitadas según el estado de la pregunta.
- Separación de pestañas por rol.
- Confirmación antes de aprobar o rechazar.
- Etiquetas textuales y colores para identificar estados.
- Paginación y filtros para reducir la cantidad de información mostrada simultáneamente.

6. Escenario de calidad: modificabilidad

6.1. Contexto

La aplicación se encuentra en ejecución como un monolito de escritorio. La información se almacena en SQLite y los usuarios interactúan mediante pestañas realizadas en swing. Se debe incorporar una nueva regla de validación, modificar una regla de negocio o sustituir un adaptador de persistencia sin alterar las vistas existentes ni duplicar lógica.

6.2. Estímulo

Se solicita agregar una nueva validación estructural para las preguntas o reemplazar el repositorio SQLite por otro repositorio compatible. Como ejemplo de extensión, se puede requerir una validación adicional sobre el contenido de los distractores.

6.3. Respuesta del sistema

- Para agregar una validación, se puede crear otra implementación de ValidadorPregunta o ampliar ValidadorEstructuralPregunta.
- PreguntaServicioImpl depende de la interfaz ValidadorPregunta, no de una implementación concreta.
- Para cambiar la persistencia, se puede crear otra implementación de PreguntaRepository o UsuarioRepository.
- Las vistas continúan comunicándose con PreguntaController.
- El controlador continúa utilizando PreguntaService.
- Las reglas de negocio no dependen directamente de sentencias SQL.
- Los repositorios SQLite encapsulan la creación de conexiones y las consultas.
- Las notificaciones se agregan mediante observadores sin modificar la lógica central del servicio.

6.4. Medición de calidad

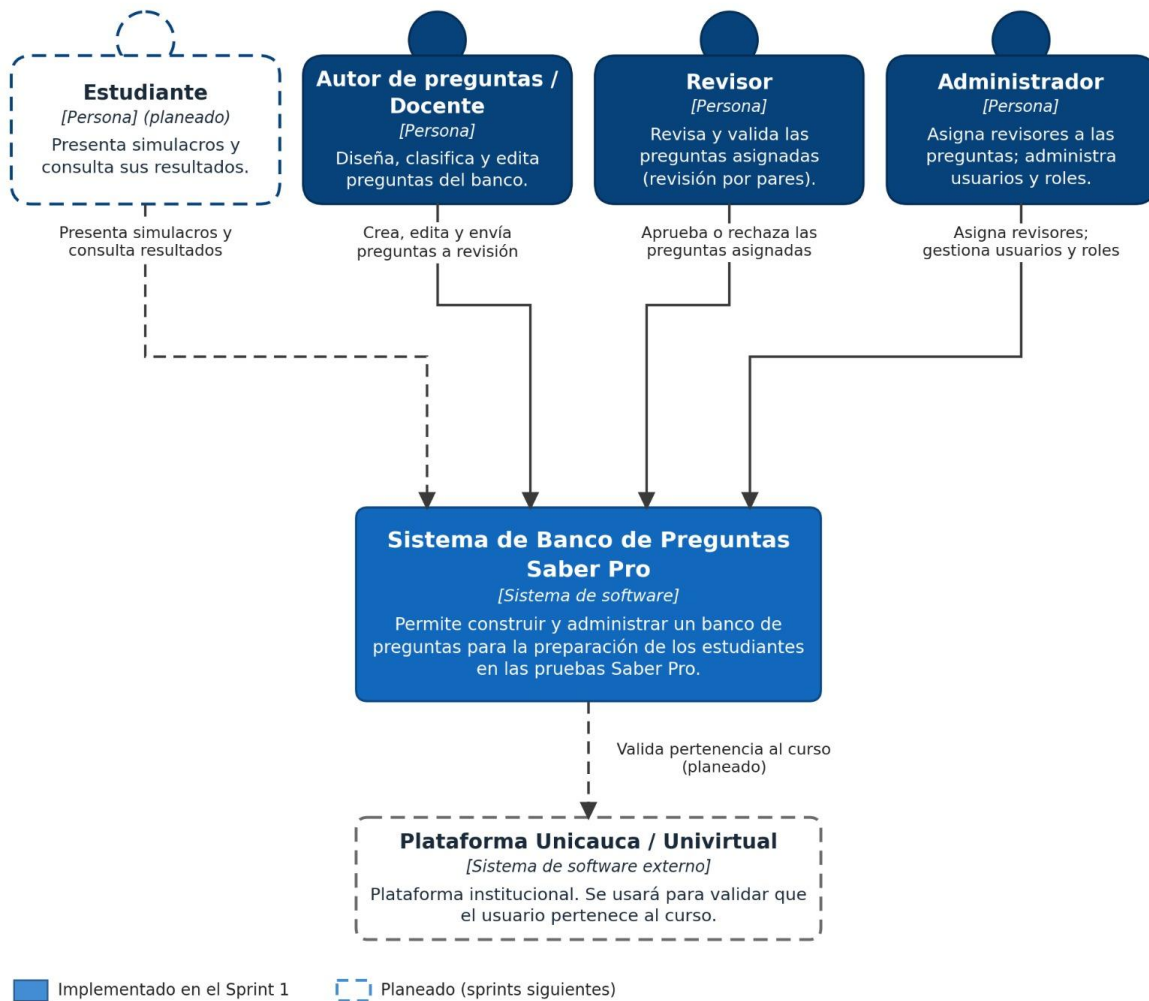
La modificabilidad se evidencia mediante:

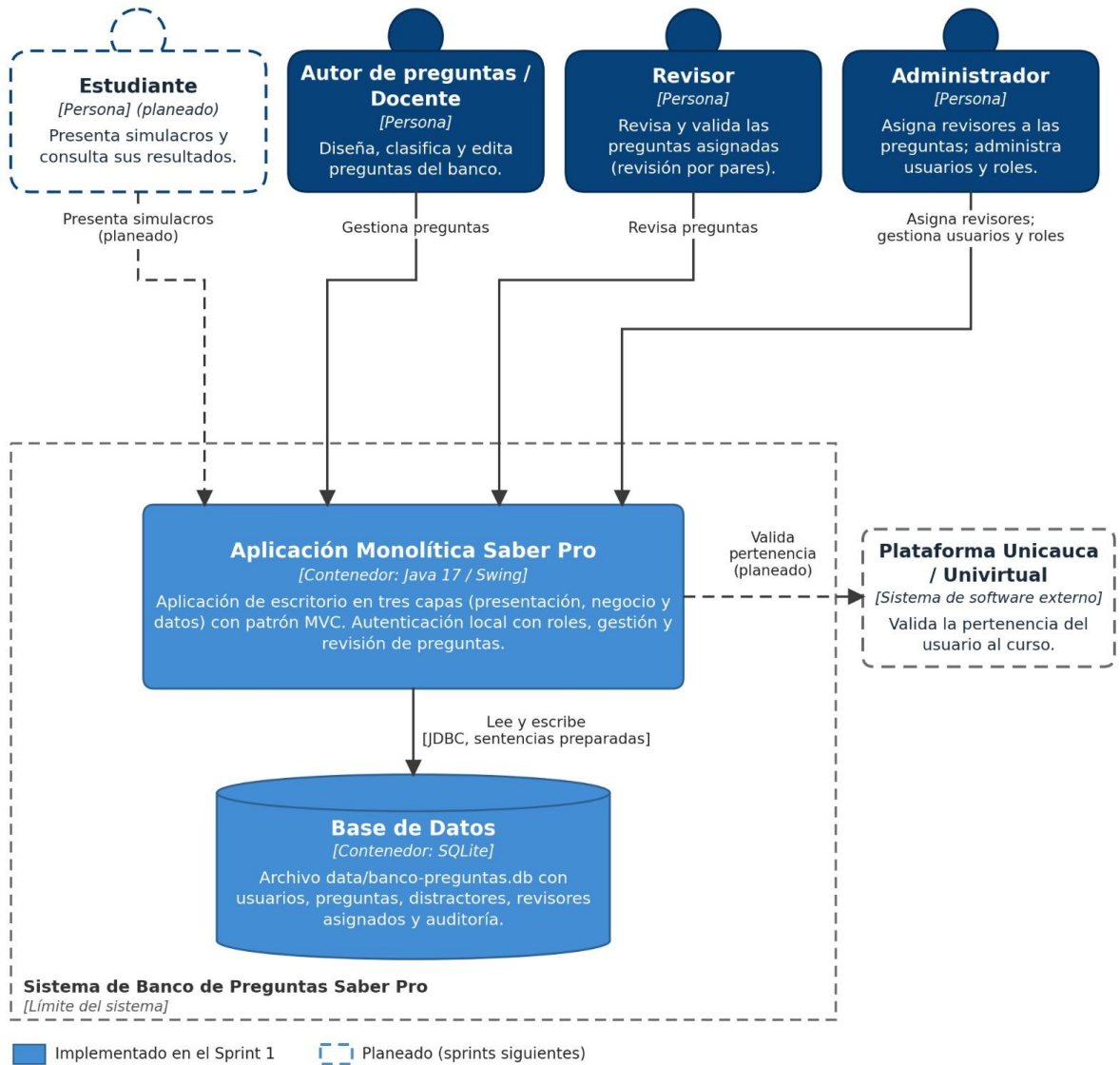
- Separación de paquetes por responsabilidad.
- Dependencia de interfaces en *PreguntaServiceImpl*.
- Inyección de dependencias desde App.
- Uso de *ValidadorPregunta* como estrategia intercambiable.
- Uso de *PreguntaRepository*, *UsuarioRepository* y *RepositorioAuditoria* como abstracciones.
- Uso de *PreguntaController* como límite entre presentación y negocio.
- Uso de *Pregunta.Builder* para construir y reconstruir el dominio.
- Pruebas con repositorios en memoria para aislar la lógica de negocio de SQLite.

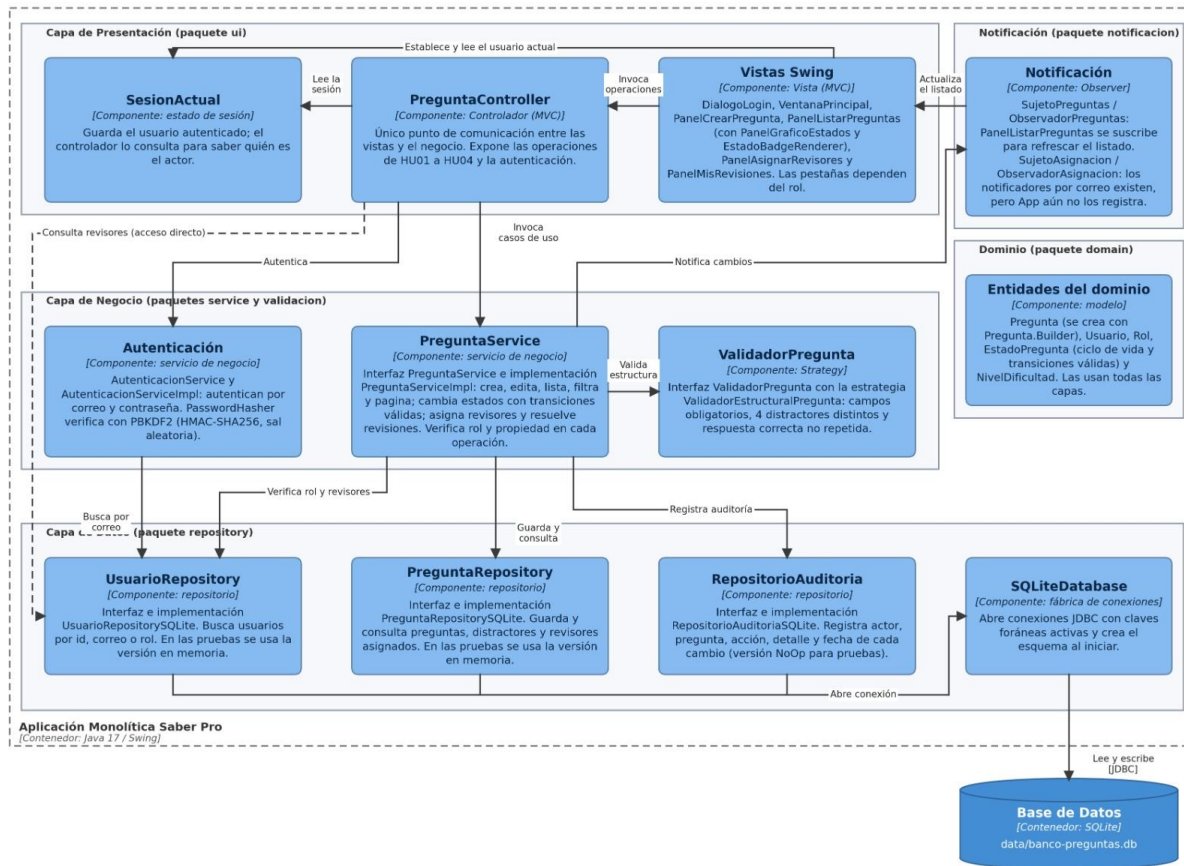
6.5. Resultado esperado

Una modificación en la estrategia de validación, en un repositorio o en un observador no debe obligar a cambiar las vistas Swing ni las entidades que no participan en la modificación. Las pruebas de *PreguntaServiceImpl* pueden ejecutar la lógica con *PreguntaRepositoryEnMemoria* y *UsuarioRepositoryEnMemoria*, lo que muestra que la lógica de negocio está desacoplada de la base SQLite.

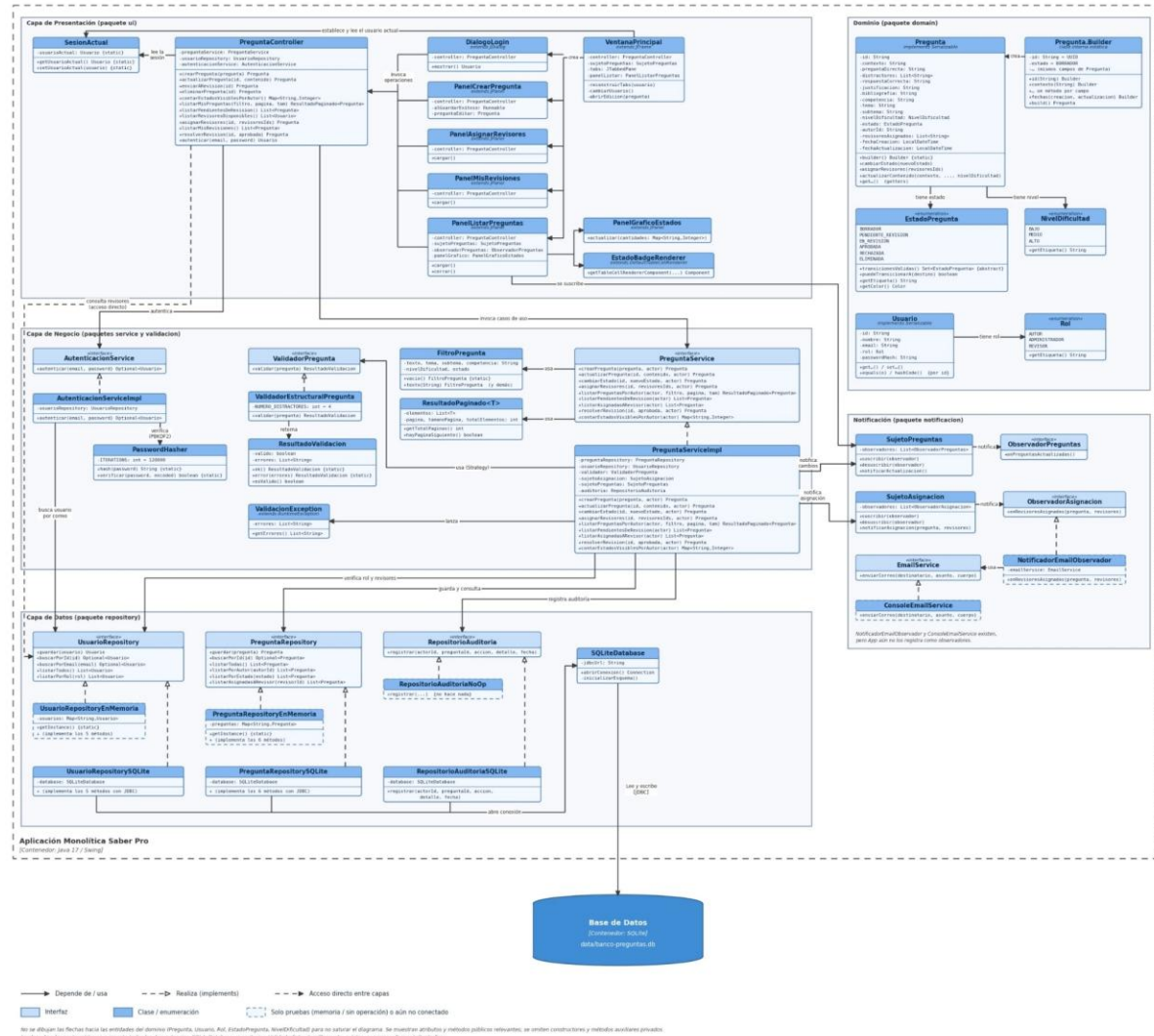
7. Arquitectura y diseño de software usando C4







→ Depende de / invoca - - -> Acceso directo entre capas
 No se dibujan las flechas hacia las entidades del dominio para no saturar el diagrama. La clase App crea e inyecta las dependencias.



8. Patrones de diseño implementados

8.1. MVC

El micro-patrón MVC se implementa mediante:

- **Vista:** paneles Swing como `PanelCrearPregunta`, `PanelListarPreguntas`, `PanelAsignarRevisores` y `PanelMisRevisiones`.
- **Controlador:** `PreguntController`.
- **Modelo:** entidades del dominio, especialmente `Pregunta`, `Usuario`, `EstadoPregunta` y el modelo interno de la tabla.

Las vistas no acceden directamente a los repositorios. Todas las operaciones pasan por el controlador.

8.2. Builder

Pregunta.Builder permite construir una pregunta paso a paso mediante métodos como ***contexto(...)***, ***preguntaDirecta(...)***, ***distractores(...)***, ***respuestaCorrecta(...)*** y ***nivelDificultad(...)***.

Se utiliza:

- En *PanelCrearPregunta* para construir preguntas desde los campos de la interfaz.
- En *App* para construir preguntas iniciales.
- En ***PreguntaRepositorySQLite.mapear(...)*** para reconstruir preguntas almacenadas, conservando id, revisores y fechas.

8.3. Strategy

ValidadorPregunta es la estrategia abstracta. *ValidadorEstructuralPregunta* es la implementación utilizada en la primera iteración.

PreguntaServiceImpl recibe un *ValidadorPregunta* por constructor. Por este motivo, la lógica de negocio puede utilizar otra estrategia de validación sin acoplarse a una clase concreta.

8.4. Observer

SujetoPreguntas mantiene observadores interesados en cambios de preguntas. *PanelListarPreguntas* se suscribe y recarga la información cuando recibe una notificación.

SujetoAsignacion notifica cuando el administrador asigna revisores. *ObservadorAsignacion* define el contrato para observadores de la asignación.

8.5. Repository

Las interfaces *PreguntaRepository*, *UsuarioRepository* y *RepositorioAuditoria* separan la lógica de negocio de la forma concreta de almacenar los datos.

Implementaciones presentes:

- *PreguntaRepositorySQLite*.
- *PreguntaRepositoryEnMemoria*.
- *UsuarioRepositorySQLite*.
- *UsuarioRepositoryEnMemoria*.
- *RepositorioAuditoriaSQLite*.
- *RepositorioAuditoriaNoOp*.

Esto permite utilizar SQLite en la aplicación y repositorios en memoria en las pruebas unitarias.

8.6. Dependency Inversion

PreguntaServiceImpl recibe por constructor interfaces de repositorio, validación y notificación.

La capa de negocio no crea directamente sus repositorios ni conoce detalles de la interfaz gráfica. Las implementaciones concretas se utilizan en el main.

9. Requerimientos funcionales y evidencia de cumplimiento

Requisito	Evidencia	Estado en la primera iteración
RF-01 Registrar usuarios	Usuario, UsuarioRepository, UsuarioRepositoryS QLite, inicialización en App	Parcial: se inicializan usuarios, no existe formulario de registro
RF-02 Autenticación	DialogoLogin, AutenticacionServiceImpl, PasswordHasher	Implementado
RF-03 Roles	Rol, VentanaPrincipal, exigirRol	Parcial: Autor, Administrador y Revisor
RF-04 Crear preguntas	PanelCrearPregunta, PreguntaServiceImpl.crear Pregunta	Implementado
RF-05 Campos de pregunta	Pregunta, Pregunta.Builder, PanelCrearPregunta	Implementado

Requisito	Evidencia	Estado en la primera iteración
RF-06 Modificar borradores	actualizarPregunta, PanelCrearPregunta	Parcial: también permite editar rechazadas por el flujo definido
RF-07 Consultar con filtros	FiltroPregunta, listarPreguntasPorAutor, PanelListarPreguntas	Implementado
RF-08 Validar contexto	ValidadorEstructuralPregunta	Implementado
RF-09 Única pregunta directa	Pregunta.preguntaDirecta	Implementado como un campo único
RF-10 Cuatro distractores	validarDistractoresYRespuesta	Implementado
RF-11 Única respuesta correcta	respuestaCorrecta como valor único y validación de no coincidencia	Parcial: no existe evaluación de múltiples respuestas
RF-12 Prohibir expresiones específicas	No hay regla	No implementado

Requisito	Evidencia	Estado en la primera iteración
RF-14 Estados del ciclo de vida	EstadoPregunta	Implementado
RF-15 Transiciones válidas	transicionesValidas, puedeTransicionar	Implementado para estados existentes
RF-16 Asignar revisores	PanelAsignarRevisores, asignarRevisores	Implementado
RF-17 Formato de evaluación	PanelMisRevisiones solo tiene aprobar/rechazar	implementado
RF-18 Observaciones de revisores	No existe atributo o tabla de observaciones	No implementado
RF-19 Historial de revisiones	Auditoría general, sin entidad de revisión	Parcial
RF-20 Aprobar o rechazar	PanelMisRevisiones, resolverRevision	Implementado
RF-29 Trazabilidad	auditoria, RepositorioAuditoriaSQLite	Implementado para acciones de preguntas
RF-30 Fecha y	actor_id, fecha, registrar(...)	Implementado

Requisito	Evidencia	Estado en la primera iteración
usuario de modificación		

10. Requerimientos no funcionales y evidencia de cumplimiento

Requisito	Evidencia	Estado en la primera iteración
RNF-01 Interfaz intuitiva y consistente	Vistas Swing, mensajes, estados por color y acciones por rol	Implementado o parcialmente
RNF-04 Consulta menor a 3 segundos	Filtros y repositorios SQLite	No existe medición de rendimiento
RNF-06 Autenticación	AutenticacionServiceImpl, DialogoLogin	Implementado
RNF-07 Restricción por rol	exigirRol, VentanaPrincipal, PreguntaController	Implementado
RNF-08 Hash seguro de contraseñas	PasswordHasher con PBKDF2-HMAC-SHA256	No Implementado

Requisito	Evidencia	Estado en la primera iteración
RNF-09 Auditoría	RepositorioAuditoriaSQLite y llamadas desde el servicio	Parcial: no se registra explícitamente el inicio de sesión
RNF-10 SOLID y diseño OO	Interfaces, servicios, repositorios y dependencias inyectadas	Implementado o parcialmente
RNF-11 Mantenimiento y nuevos módulos	Paquetes separados e interfaces	Implementado o parcialmente
RNF-12 Separación de capas	ui, service, domain, repository	Implementado o
RNF-13 Nuevos tipos y módulos	Strategy, Repository, Builder y dependencias por interfaces	Implementado o parcialmente
RNF-14 Bajo acoplamiento y alta cohesión	Controlador, servicios, repositorios y sujetos separados	Implementado o parcialmente
RNF-15 Integridad de información	Transacciones, claves foráneas y validación previa	Implementado o

11. Pruebas automatizadas relacionadas con la arquitectura

11.1. Validación

[src/test/java/com/bancopreguntas/validacion/ValidadorEstructuralPreguntaTest.java](#) verifica:

- Pregunta completa válida.
- Contexto obligatorio.
- Exactamente cuatro distractores.
- Distractores diferentes.
- Respuesta correcta no repetida.
- Nivel de dificultad obligatorio.
- Acumulación de múltiples errores.

11.2. Dominio

[src/test/java/com/bancopreguntas/domain/PreguntaTest.java](#) verifica:

- Estado inicial de borrador.
- Generación de id y fechas.
- Cantidad de distractores.
- Inmutabilidad de listas expuestas.
- Actualización del estado y fecha.
- Asignación de revisores.

[src/test/java/com/bancopreguntas/domain/EstadoPreguntaTest.java](#) verifica:

- Colores para los estados.
- Transiciones válidas.
- Rechazo de transiciones inválidas.
- Ausencia de transiciones desde APROBADA.

11.3. Servicio de negocio

[src/test/java/com/bancopreguntas/service/PreguntaServiceImplTest.java](#) verifica:

- Creación válida e inválida.
- Transiciones de estado.
- Paginación.
- Filtros.
- Separación de preguntas por autor.
- Asignación de revisores.
- Validación de roles.
- Edición por propietario.
- Edición de preguntas rechazadas.

- Consulta de preguntas asignadas.
- Cambio de “en revisión” a “revisando”
- Conteo de estados.
- Fallo seguro de consultas sin actor autenticado.

11.4. Persistencia

[src/test/java/com/bancopreguntas/repository/SQLitePersistenceTest.java](#) verifica:

- Persistencia de preguntas, distractores, revisores y fechas.
- Consulta de preguntas asignadas.
- Autenticación mediante hash.
- Rechazo de credenciales incorrectas.

12. Github URL

[Banco de preguntas](#)

13. YouTube URL

[Video](#)